GAZİ ÜNİVERSİTESİ

Gazi

1926

DERİN PEKİŞTİRMELİ ÖĞRENME TABANLI ÇOKLU AJANLI TAKİP-KAÇMA SİSTEMİ

(İşyeri Eğitimi Projesi)

Bedirhan ÇELİK

GAZİ ÜNİVERSİTESİ TEKNOLOJİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

OCAK 2026

ÖZET

Bu çalışmada, çoklu ajan sistemleri ve derin pekiştirmeli öğrenme algoritmalarının dinamik ortamlardaki performansını incelemek amacıyla özgün bir takip-kaçma(pursuit-evasion) simülasyon ortamı geliştirilmiştir. Projenin temel amacı, otonom ajanların karmaşık senaryolarda işbirliği yapma ve stratejik karar alma yeteneklerinin analiz edilmesidir. Çalışma kapsamında, Python programlama dili ve Pygame kütüphanesi kullanılarak iki boyutlu, sürekli bir fiziksel çevre modellenmiştir.

Araştırmada yöntem olarak, ajanların karar mekanizmalarını oluşturmak için Derin Q-Ağları (DQN) ve Proksimal Politika Optimizasyonu (PPO) gibi modelden bağımsız pekiştirmeli öğrenme teknikleri kullanılmıştır. Ajanların çevre algısı, Lidar benzeri ışın izleme (ray-casting) sensör simülasyonları ile sağlanmıştır. Eğitim sürecinin verimliliğini artırmak ve ajanların öğrenme hızını optimize etmek amacıyla, zorluk seviyesinin kademeli olarak artırıldığı müfredata dayalı öğrenme (curriculum learning) stratejileri sisteme entegre edilmiştir.

Geliştirilen modelin doğrulanması, farklı engel yoğunluklarına ve ajan sayılarına sahip test senaryoları üzerinden gerçekleştirilmiştir. Simülasyon sonuçları, ajanların kümülatif ödül grafikleri, kazanma/kaybetme oranları ve bölüm uzunlukları üzerinden değerlendirilmiştir. Elde edilen bulgular, geliştirilen algoritmaların ajanlara başarılı bir şekilde işbirlikçi takip ve etkili kaçınma manevraları kazandırdığını göstermiştir. Çalışma sonucunda, tasarlanan simülasyon ortamının otonom sistemlerin eğitimi için geçerli ve esnek bir altyapı sunduğu tespit edilmiştir.

Anahtar Kelimeler : Pekiştirmeli Öğrenme, Çoklu Ajan Sistemleri, Takip-Kaçma, Simülasyon, Yapay Sinir Ağları, Otonom Robotik.

Sayfa Adedi : 31

DEEP REINFORCEMENT LEARNING BASED MULTI-AGENT PURSUIT-EVASION SYSTEM

(Industrial Training Project)

Bedirhan ÇELİK

GAZI UNIVERSITY FACULTY OF TECHNOLOGY

DEPARTMENT OF COMPUTER ENGINEERING

JANUARY 2026

ABSTRACT

In this study, a unique pursuit-evasion simulation environment has been developed to investigate the performance of multi-agent systems and deep reinforcement learning algorithms in dynamic environments. The primary objective of the project is to analyze the capabilities of autonomous agents regarding cooperation and strategic decision-making within complex scenarios. Within the scope of the study, a two-dimensional continuous physical environment was modelled using the Python programming language and the Pygame library.

As the research methodology, model-free reinforcement learning techniques, specifically Deep Q-Networks (DQN) and Proximal Policy Optimization (PPO), were utilized to construct the decision-making mechanisms of the agents. The environmental perception of the agents was provided through Lidar-like ray-casting sensor simulations. To enhance the efficiency of the training process and optimize the learning rate of the agents, curriculum learning strategies, in which the difficulty level is gradually increased, were integrated into the system.

The validation of the developed model was carried out through test scenarios with varying obstacle densities and agent counts. Simulation results were evaluated based on cumulative reward graphs, win/loss ratios, and episode lengths. The obtained findings demonstrated that the developed algorithms successfully endowed the agents with cooperative pursuit and effective evasion maneuvers. Consequently, it was determined that the designed simulation environment offers a valid and flexible infrastructure for the training of autonomous systems.

Key Words : Reinforcement Learning, Multi-Agent Systems, Pursuit-Evasion, Simulation, Artificial Neural Networks, Autonomous Robotics.

Page Number : 31

TEŞEKKÜR

Bu projenin tasarım, geliştirme ve raporlama süreçleri, benim için teknik yetkinliklerimi derinleştirdiğim son derece öğretici bir deneyim olmuştur. Özellikle çoklu ajan sistemleri ve pekiştirmeli öğrenme algoritmalarının simülasyon ortamına entegrasyonu sırasında karşılaşılan karmaşık problemler, proje sürecini zorlu kılsa da, bu zorlukların aşılması mesleki gelişimime önemli katkılar sağlamıştır.

Bu çalışmanın gerçekleştirilmesinde bana imkan sağlayan, kurumsal kültürü ve sunduğu çalışma ortamı ile projenin hayata geçirilmesine zemin hazırlayan HAVELSAN'a teşekkür ederim.

Projenin her aşamasında teknik bilgi ve tecrübeleriyle bana yol gösteren, karşılaştığım problemlerde değerli çözüm önerileri sunan ve çalışmanın başarıyla tamamlanmasında büyük emeği geçen Sayın *Ahmet Gökhan GÖZE*'ye, Sayın *Melih Can HAMURCU*'ya ve takım liderimiz Sayın *Çağlar AKMAN*'a rehberlikleri ve katkıları için şükranlarımı sunarım.

Ayrıca, lisans eğitimim süresince edindiğim teorik altyapının bu projeye aktarılmasındaki katkıları, akademik vizyonu ve süreç boyunca esirgemediği değerli yönlendirmeleri için danışman hocam Sayın Prof. Dr. Hüseyin POLAT'a en içten teşekkürlerimi sunarım.

İÇİNDEKİLER

Sayfa

ÖZET iv

ABSTRACT v

TEŞEKKÜR vi

İÇİNDEKİLER vii

ÇİZELGELERİN LİSTESİ ix

ŞEKİLLERİN LİSTESİ ix

RESİMLERİN LİSTESİ ix

HARİTALARIN LİSTESİ ix

SİMGELER VE KISALTMALAR x

1. GİRİŞ 1

2. İŞYERİ HAKKINDA GENEL BİLGİLER 3

2.1. İşyerinin Tarihçesi ve Faaliyet Alanları 3

2.2. Organizasyon Yapısı ve Personel Profili 4

2.3. İşyeri Birimleri 5

2.3.1. Komuta Kontrol ve Savunma Teknolojileri 5

2.3.2. Simülasyon, Otonom ve Platform Yönetim Teknolojileri 5

2.3.3. Bilgi ve İletişim Teknolojileri 5

2.3.4. Ülke Güvenliği ve Siber Güvenlik 5

2.4. Yürütülen Başlıca Projeler 6

3. İŞYERİNDE GÖREV ALINAN ÇALIŞMALAR 7

3.1. Literatür Araştırması ve Simülasyon Altyapısının Analizi 7

3.1.1. Literatür taraması: MARL ve takip-kaçış problemi 7

3.1.2. Geliştirme ortamı ve teknik stack 7

3.1.3. Vektörel fizik motoru analizi ve geliştirme 8

3.2. Yapay Zeka Modelinin Geliştirilmesi Ve Optimizasyonu 8

3.2.1. DQN algoritması ve matematiksel modeli 8

3.2.2 Ağ mimarisi karşılaştırmaları: LSTM ve dense layer 10

3.2.3. Hiperparametre optimizasyonu 10

3.2.4. Stabilite çalışmaları (PER ve soft update) 11

3.3. Ödül Fonksiyonu Mühendisliği ve Ortam İyileştirmeleri 13

3.4. Test Ve Doğrulama Altyapısı: İnteraktif Senaryo Oluşturucu (GUI) 14

3.4.1 Tasarım modu ve dinamik çevre kurgusu 14

3.4.2. Veri kalıcılığı ve JSON serileştirme 14

3.4.3. Hibrit kontrol mekanizması 15

3.4.4. Görselleştirme ve raporlama modülü 15

3.4.5 Performans analizi 17

3.5 Donanım Uyumluluğu Ve Gerçek Zamanlılık Çalışmaları 17

3.5.1. Simülasyon zaman adımı (dt) uyarlaması 17

3.5.2. Frame skipping (kare atlama) yöntemi 17

3.6 Fizik Motoru Değişimi Ve İleri Simülasyon Altyapısı (MuJoCo) 18

3.6.1. Hidrodinamik modelleme ve hareket denklemleri 18

3.6.2. Stable Baselines3 entegrasyonu ve PPO algoritması 19

3.6.3. Genişletilebilir modüler mimari: Kara, hava ve deniz entegrasyonu 20

4. PROJE KONUSU 21

4.1. Giriş 21

4.2. Yöntem ve Teknikler 22

4.2.1. Problemin matematiksel modeli (POMDP) 22

4.2.2. Sistem mimarisi ve karar modülü..... 25

4.3. Bulgular..... 26

4.3.1. Eğitim stabilitesi ve algoritma analizi)..... 26

4.3.2. Ödül şekillendirmenin davranışsal etkisi 26

4.3.3. Kinematik performans ve rota verimliliği 27

5. SONUÇ VE ÖNERİLER 29

KAYNAKLAR 31

EKLER 32

EK-1. İşyeri haftalık çalışma raporu 33

EK-2 Anket formları..... 34

ÖZGEÇMİŞ 37

ÇİZELGELERİN LİSTESİ

Çizelge **Sayfa**

Çizelge 4.1. DQN yakalama oranı grafiği..... 27

ŞEKİLLERİN LİSTESİ

Şekil **Sayfa**

Şekil 2.1. İşyeri organizasyon şeması 4

Şekil 3.1 Dense Layer 10

Şekil 3.2 LSTM..... 10

Şekil 4.1 POMDP..... 22

RESİMLERİN LİSTESİ

Resim **Sayfa**

Resim 2.1. İşyeri dış görünümü 4

Resim 3.1. İnteraktif Senaryo Oluşturucu Tasarım Modu..... 9

Resim 3.2 İnteraktif Senaryo Oluşturucu Çalıştırma Modu 16

Resim 3.3 Mujoco Deniz Ortamı..... 18

HARİTALARIN LİSTESİ

Harita **Sayfa**

Harita 2.1. İşyerinin konum haritası..... 3

SİMGELER VE KISALTMALAR

SİMGELER VE KISALTMALAR

Bu çalışmada kullanılmış simgeler ve kısaltmalar, açıklamaları ile birlikte aşağıda sunulmuştur.

Simgeler	Açıklamalar
dt	Simülasyon zaman adımı (Time step)
ε	Epsilon (Keşif oranı / Exploration rate)
γ	Gamma (İndirim faktörü / Discount factor)
%	Yüzde işareti
α	Alpha (Öğrenme oranı / Learning Rate)

Kısaltmalar	Açıklamalar
AI	Artificial Intelligence (Yapay Zeka)
DQN	Deep Q-Network (Derin Q-Ağı)
PPO	Proximal Policy Optimization (Proksimal Politika Optimizasyonu)
A2C	Advantage Actor Critic (Avantajlı Oyuncu Eleştirmen)
FPS	Frames Per Second (Saniyedeki Kare Sayısı)
GUI	Graphical User Interface (Grafiksel Kullanıcı Arayüzü)
JSON	JavaScript Object Notation (Veri değişim formatı)
LSTM	Long Short-Term Memory (Uzun Kısa Süreli Bellek)
MARL	Multi-Agent Reinforcement Learning (Çoklu Ajan Pekiştirmeli Öğrenme)
PER	Prioritized Experience Replay (Önceliklendirilmiş Deneyim Tekrarı)
RL	Reinforcement Learning (Pekiştirmeli Öğrenme)
TSKGV	Türk Silahlı Kuvvetlerini Güçlendirme Vakfı
AR-GE	Araştırma-Geliştirme
MuJoCo	Multi-Joint dynamics with Contact

1. GİRİŞ

Bu proje çalışması, Türk savunma sanayisinin ve bilişim sektörünün öncü kuruluşlarından biri olan HAVELSAN bünyesinde, Modelleme ve Sensör Teknolojileri biriminde gerçekleştirilmiştir. Staj süresince, günümüz mühendislik uygulamalarında giderek artan bir öneme sahip olan yapay zeka destekli otonom sistemler ve bu sistemlerin simülasyon ortamlarında eğitilmesi süreçleri üzerine odaklanılmıştır. Teknolojinin gelişmesiyle birlikte, insansız deniz araçlarının tekil kullanımdan ziyade sürü halinde veya çoklu ajan sistemleri olarak görev yapması ihtiyacı doğmuştur. Ancak, birden fazla otonom ajanın dinamik, engellerle dolu ve belirsiz bir ortamda, ortak bir hedefe yönelik işbirliği içinde hareket etmesini sağlamak karmaşık bir optimizasyon problemidir. Klasik kural tabanlı algoritmalar, öngörülemeyen senaryolarda yetersiz kalabilmekte ve her durum için ayrı kodlama gerektirmektedir. Bu problemin çözümüne yönelik olarak gerçekleştirilen bu projede, ajanların çevreyle etkileşime girerek deneme-yanılma yoluyla strateji geliştirmesini sağlayan "Pekiştirmeli Öğrenme" yöntemleri ele alınmıştır.

Çalışmanın Konusu ve Problem Tanımı

Proje kapsamında ele alınan temel problem, literatürde "Takip-Kaçış" (Pursuit-Evasion) olarak bilinen, bir veya birden fazla takipçi ajanın kaçan bir hedefi engellerle dolu bir ortamda yakalamaya çalıştığı senaryodur. Mevcut literatür incelendiğinde, bu problemin çözümünde genellikle deterministik ve kural tabanlı yaklaşımların kullanıldığı, ancak bu yöntemlerin dinamik çevre koşullarında esneklik gösteremediği görülmektedir. Özellikle savunma sanayii uygulamalarında, sahadaki belirsizlikler ve anlık değişen tehditler, sistemlerin otonom olarak yeni stratejiler geliştirmesini zorunlu kılmaktadır. Araştırmada, "Seyrek Ödül" (Sparse Reward) problemi olarak adlandırılan, ajanın hedefe ulaşana kadar doğru yolda olup olmadığına dair geri bildirim alamaması durumu ve "Keşif-Sömürü" (Exploration-Exploitation) dengesinin kurulması gibi zorluklar üzerine yoğunlaşılmıştır. Bu bağlamda, Çoklu Ajan Pekiştirmeli Öğrenme (MARL) teknikleri ve Derin Sinir Ağları (DNN) entegrasyonu, problemin çözümü için temel dayanak noktasını oluşturmaktadır.

Araştırmanın Amacı ve Önemi

Araştırmanın temel amacı, otonom ajanların takip-kaçma senaryolarında, önceden tanımlanmış kurallar olmaksızın, tamamen görsel ve vektörel algılarına dayanarak akıllı kararlar verebilmesini sağlayan yüksek sadakatli bir simülasyon altyapısı geliştirmektir. Bu kapsamda, ajanların eğitimi için Derin Q-Ağları (DQN) ve Proksimal Politika Optimizasyonu (PPO) gibi modern algoritmaların sisteme entegrasyonu ve hiperparametre optimizasyonları hedeflenmiştir. Çalışmanın önemi, savunma sanayisi başta olmak üzere, robotik sistemlerin maliyetli ve riskli gerçek dünya testleri yerine, fiziksel gerçekçiliği yüksek simülasyon ortamlarında güvenle eğitilebileceğini göstermesinden kaynaklanmaktadır. Geliştirilen bu "Simülasyondan Gerçeğe" (Sim-to-Real) altyapısı, İnsansız Deniz Araçlarının (İDA) operatör müdahalesi olmadan karmaşık manevraları öğrenebilmesine olanak tanıyarak, operasyonel maliyetlerin düşürülmesine ve görev başarısının artırılmasına katkı sağlama potansiyeli taşımaktadır.

Araştırmanın Yöntemi ve Kapsamı

Projenin gerçekleştirilmesinde iteratif bir yazılım geliştirme ve modelleme yöntemi izlenmiştir. İlk aşamada Python programlama dili ve Pygame kütüphanesi kullanılarak vektörel tabanlı özgün bir fiziksel çevre modellenmiş; daha sonraki aşamalarda fiziksel gerçekçiliği artırmak amacıyla MuJoCo fizik motoruna geçiş yapılmıştır. Ajanların karar mekanizması için yapay sinir ağları tasarlanmış, öğrenme sürecini stabilize etmek adına "Prioritized Experience Replay" ve "Polyak Averaging" gibi literatürde kabul görmüş teknikler uygulanmıştır. Araştırma kapsamı, İDA'ların su üstü hareket dinamikleri ve engel sakınma manevraları ile sınırlandırılmış olup, geliştirilen algoritmaların performansı "Capture Rate" (Yakalama Oranı) ve "Average Reward" (Ortalama Ödül) metrikleri üzerinden analiz edilmiştir.

Araştırmanın Varsayımları

Proje sürecinde, araştırmacının doğrudan kontrolü dışında kalan ve çalışmanın geçerliliğini etkileyen bazı dışsal faktörler sabit ve doğru kabul edilmiştir. Bu kapsamda;

- Çalışmada kullanılan açık kaynak kodlu kütüphanelerin (PyTorch, NumPy, Stable Baselines3) temel matematiksel işlemleri (türev alma, matris çarpımı, gradyan hesaplama) hatasız gerçekleştirdiği varsayılmıştır.

- Simülasyon ortamında kullanılan MuJoCo fizik motorunun, suyun kaldırma kuvveti ve sürtünme katsayıları gibi hidrodinamik hesaplamaları, gerçek dünya fiziğini istatistiksel olarak anlamlı bir düzeyde temsil ettiği kabul edilmiştir.

- Eğitimlerin gerçekleştirildiği donanım altyapısının (GPU ve CPU), derin sinir ağlarının yakınsaması sürecinde deterministik olmayan (stokastik) hatalar üretmediği ve hesaplama gücünün stabil olduğu varsayılmıştır.

Araştırmanın Sınırlılıkları

Yapılan araştırma ve geliştirilen simülasyon ortamı, kapsam ve yöntem açısından belirli sınırlılıklara sahiptir. Bu sınırlılıklar kavramsal ve yöntemsel olmak üzere iki grupta ele alınabilir.

Kavramsal Sınırlılıklar: Çalışma, gerçek dünyanın sonsuz karmaşıklığını hesaplanabilir bir düzeye indirgemek adına, proje başlangıcında iki boyutlu (2D) bir düzlem ile sınırlandırılmıştır; her ne kadar son aşamada 3D ortama geçilse de, hava durumu değişimleri (fırtına, yağmur) gibi çevresel gürültü faktörleri kapsam dışı bırakılmıştır. Ayrıca, ajanların iletişim modelleri, doğrudan veri paylaşımı veya stratejik haberleşme yerine, sadece fiziksel konum ve sensör verileri üzerinden dolaylı etkileşim ile sınırlandırılmıştır.

Yöntemsel Sınırlılıklar: Ajanların hareket ettiği fiziksel uzay sürekli (continuous) yapıda olmasına rağmen, kullanılan DQN algoritmasının yapısı gereği karar uzayı (aksiyonlar) belirli açılara bölünerek ayrıklaştırılmıştır; bu durum robot hareketlerinde mikroskobik düzeyde keskinliklere neden olabilmektedir.

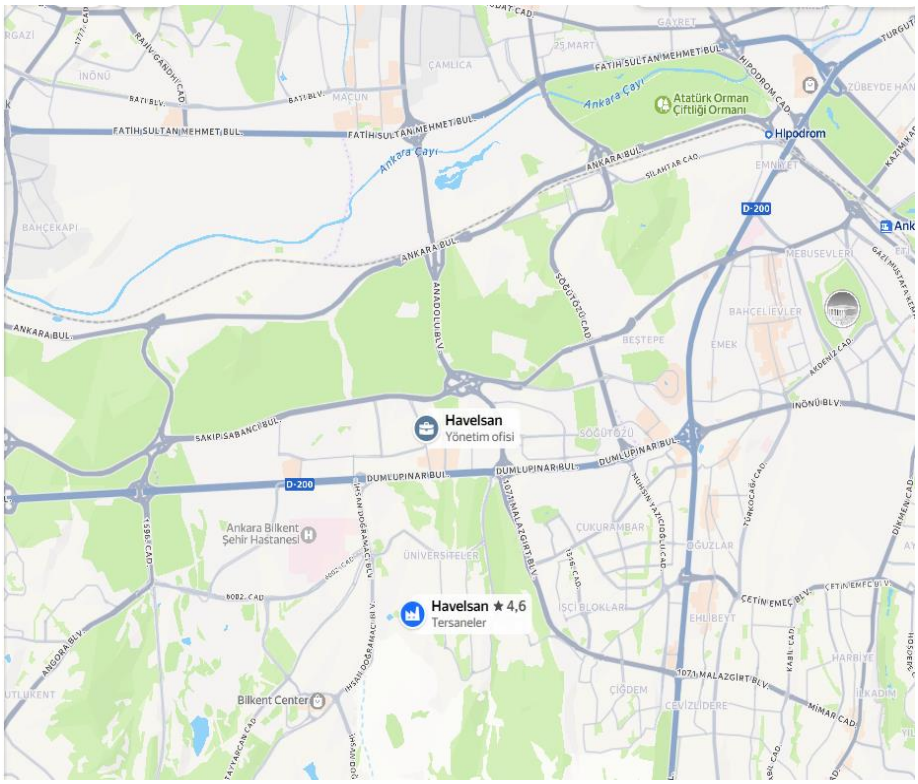
2. İŞYERİ HAKKINDA GENEL BİLGİLER

Bu bölümde, staj eğitiminin gerçekleştirildiği HAVELSAN (Hava Elektronik Sanayi) firmasına ait tarihçe, organizasyon yapısı, faaliyet alanları ve yürütülen temel projelere ilişkin genel bilgilere yer verilmiştir.

2.1. İşyerinin Tarihçesi ve Faaliyet Alanları

HAVELSAN, Türk Silahlı Kuvvetleri'nin modernizasyon ihtiyacının karşılanması ve savunma sanayisinde yerli katkı payının artırılması amacıyla 1982 yılında kurulmuştur [1,3]. Türk Silahlı Kuvvetlerini Güçlendirme Vakfı'na (TSKGV) bağlı bir şirket olarak kurulan işletme, faaliyetlerine ilk olarak askeri sistemlerin bakım ve onarımı ile başlamış, ilerleyen yıllarda stratejik bir dönüşüm geçirerek Türkiye'nin en büyük yazılım ve sistem entegrasyon firmalarından biri haline gelmiştir [3].

Şirket, savunma sanayisi başta olmak üzere; komuta kontrol sistemleri, simülasyon teknolojileri, ülke güvenliği, siber güvenlik ve bilgi yönetim sistemleri alanlarında faaliyet göstermektedir. Sadece askeri alanda değil, kamu ve özel sektör için geliştirilen sivil teknolojilerle de hem ulusal hem de uluslararası pazarda yetkinlik sahibidir. Şirketin genel merkezi Ankara'da bulunmakta olup, İstanbul ve çeşitli yurtdışı lokasyonlarında da ofisleri mevcuttur [1].



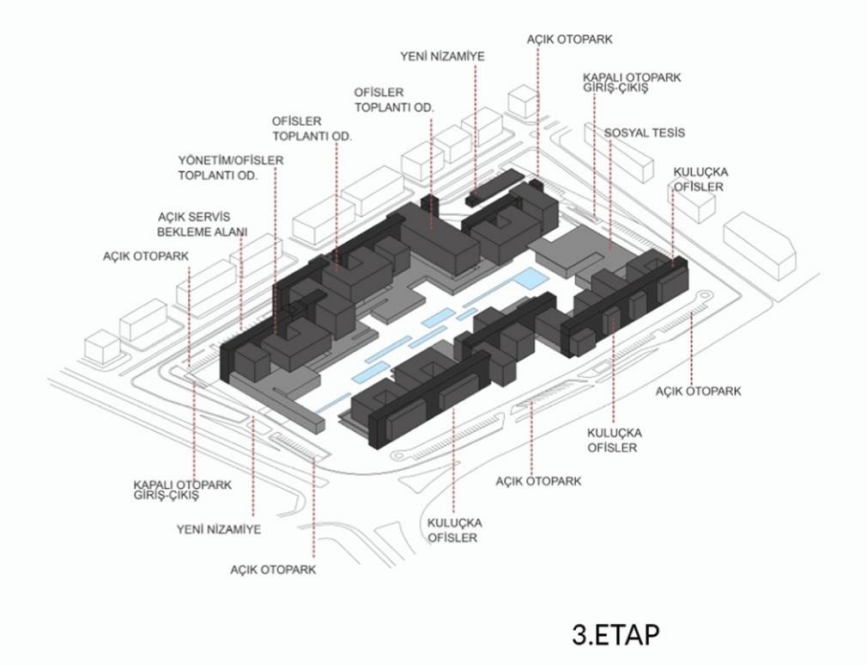
Harita 2.1: İşyerinin konum haritası [2]

2.2. Organizasyon Yapısı ve Personel Profili

HAVELSAN, %99'dan fazla hisse oranıyla Türk Silahlı Kuvvetlerini Güçlendirme Vakfı'na aittir. İşletmenin yönetim yapısı, Yönetim Kurulu ve Genel Müdürlük makamı altında yapılandırılan Genel Müdür Yardımcılıkları ve bunlara bağlı direktörlükler/grup başkanlıkları şeklindedir [3].

Resim 2.1 : İşyeri dış görünümü [2]

Yüksek teknoloji üretimi odaklı bir firma olması sebebiyle, personel profilinin büyük bir çoğunluğunu mühendisler oluşturmaktadır. Şirket bünyesinde 2500'ü aşkın personel görev yapmaktadır. Çalışanların önemli bir kısmı Bilgisayar, Elektrik-Elektronik, Makine ve Endüstri Mühendisliği gibi teknik disiplinlerden mezun olup, birçoğu yüksek lisans ve doktora derecesine sahiptir. Şirket, AR-GE faaliyetlerine büyük önem vermekte ve üniversite-sanayi işbirliği kapsamında teknokentlerde de aktif rol oynamaktadır [3].



Şekil 2.1: İşyeri organizasyon şeması [2]

2.3. İşyeri Birimleri

Şirket, uzmanlık alanlarına göre dört ana faaliyet alanında (Genel Müdür Yardımcılığı seviyesinde) yapılanmıştır.

2.3.1. Komuta Kontrol ve Savunma Teknolojileri

Bu birim, Kara, Deniz, Hava ve Uzay platformlarına yönelik ağ destekli Savaş Yönetim Sistemleri (SYS) geliştirmektedir. Milli gemi projeleri (MİLGEM), denizaltı modernizasyonları ve hava komuta kontrol sistemleri bu birimin ana sorumluluk alanındadır [3].

2.3.2. Simülasyon, Otonom ve Platform Yönetim Teknolojileri

Staj çalışmasının yürütüldüğü bu birim, şirketin en köklü faaliyet alanlarından biridir. Birim bünyesinde; askeri ve sivil uçuş simülatörleri, paraşüt simülatörleri, araç sürüş eğitim simülatörleri ve bakım eğitim sistemleri tasarlanıp üretilmektedir.

Ayrıca son yıllarda bu birim altında "Robotik ve Otonom Sistemler" çalışmalarına ağırlık verilmiştir. "Dijital Birlik" konsepti çerçevesinde İnsansız Kara Araçları (İKA), İnsansız Hava Araçları (İHA) ve İnsansız Deniz Araçları (İDA) geliştirilmekte; bu araçların sürü zekasıyla otonom olarak görev yapabilmesi üzerine AR-GE çalışmaları yürütülmektedir. Proje kapsamında incelenen çoklu ajan sistemleri ve pekiştirmeli öğrenme çalışmaları da bu vizyonun bir parçasıdır [3].

2.3.3. Bilgi ve İletişim Teknolojileri

Kamu kurumlarının ve büyük ölçekli işletmelerin dijital dönüşüm süreçlerine liderlik eden bu birim; Kurumsal Kaynak Yönetimi (ERP) sistemleri, e-Devlet çözümleri, biyometrik veri sistemleri ve görüntü işleme teknolojileri üzerine çözümler sunmaktadır [3].

2.3.4. Ülke Güvenliği ve Siber Güvenlik

Kritik altyapıların güvenliği, sınır güvenliği sistemleri, boru hattı güvenliği ve siber tehditlere karşı savunma mekanizmaları (Kalkan, Gözcü vb.) bu birim tarafından geliştirilmektedir [3].

2.4. Yürütülen Başlıca Projeler

HAVELSAN tarafından yürütülen ve kamuoyu ile paylaşılan bazı stratejik projeler şunlardır [1,3]:

- *ADVENT*: Yeni nesil, ağ destekli Veri Entegre Savaş Yönetim Sistemi.

- *Milli Muharip Uçak (KAAN) Simülatörü*: Türkiye'nin 5. nesil savaş uçağı projesi için mühendislik ve eğitim simülatörlerinin geliştirilmesi.

- *BARKAN, BAHA ve SANCAR:* Sırasıyla İnsansız Kara, Hava ve Deniz araçlarından oluşan, otonom ve sürü halinde hareket edebilen "Dijital Birlik" ailesi.

- *HEÇS:* Hava Kuvvetleri Bilgi Sistemi projesi ile hava kuvvetlerinin tüm yönetim süreçlerinin dijitalleştirilmesi.

3. İŞYERİNDE GÖREV ALINAN ÇALIŞMALAR

İşyeri eğitimi süresince, HAVELSAN Modelleme ve Sensör Teknolojileri biriminde yürütülen "Derin Pekiştirmeli Öğrenme ile Otonom Takip-Kaçış Sisteminin Geliştirilmesi" projesi kapsamında gerçekleştirilen çalışmalar bu bölümde detaylandırılmıştır. Süreç; teorik altyapının kurulmasından başlayıp, vektörel tabanlı 2 boyutlu (2D) simülasyonların geliştirilmesi, derin sinir ağlarının (DQN, PPO) optimizasyonu, gerçek zamanlı donanım uyumluluk çalışmaları ve nihayetinde yüksek sadakatli 3 boyutlu (3D) fizik motoruna geçiş aşamalarını kapsamaktadır.

3.1. Literatür Araştırması ve Simülasyon Altyapısının Analizi

Projenin başlangıç aşamasında, problemin teorik temellerini oluşturmak amacıyla Pekiştirmeli Öğrenme (RL) ve Çoklu Ajan Pekiştirmeli Öğrenme (Multi-Agent RL) konularında detaylı bir literatür taraması yapılmıştır. Özellikle Takip-Kaçış probleminin oyun teorisi ve savunma sanayii uygulamalarındaki yeri incelenmiştir.

Bu teorik altyapının üzerine, projenin teknik gereksinimleri olan Durum (State), Aksiyon (Action) ve Ödül (Reward) mekanizmaları tanımlanmıştır. Geliştirme ortamı için PyTorch, NumPy ve Pygame kütüphanelerini içeren Python tabanlı bir altyapı kurulmuştur.

3.1.1. Literatür taraması: marl ve takip-kaçış problemi

Projenin başlangıcında, Takip-Kaçış (Pursuit-Evasion) probleminin çözümünde kullanılan yöntemler incelenmiştir. Literatürde bu problem genellikle "Oyun Teorisi" veya "Potansiyel Alanlar" (Potential Fields) yöntemleriyle çözülse de, bu yaklaşımların dinamik ve belirsiz ortamlarda (Stokastik Çevreler) yetersiz kaldığı tespit edilmiştir. Bu nedenle, Çoklu Ajan Pekiştirmeli Öğrenme (MARL - Multi-Agent Reinforcement Learning) yöntemleri üzerine yoğunlaşılmıştır [10, 12]. Özellikle DQN (Deep Q-Network) mimarisi ve sürekli aksiyon uzaylarında etkili olan PPO (Proximal Policy Optimization) algoritmalarının matematiksel altyapısı (Bellman Denklemleri, Policy Gradient Teoremi) incelenmiştir [4, 6, 8].

3.1.2. Geliştirme ortamı ve teknik stack

Yazılım geliştirme süreci için endüstriyel standartlara uygun bir Python sanal ortamı (virtual environment) kurulmuştur. Kullanılan kütüphaneler ve seçim gerekçeleri şöyledir:

- *NumPy*: Yüksek boyutlu matris çarpımları ve vektör operasyonları (Dot product, Cross product) için.

- *PyTorch*: Dinamik hesaplama grafiği (Computational Graph) özelliği sayesinde, sinir ağlarının geriye yayılım (Backpropagation) işlemlerini optimize etmek için.

- *Pygame*: Simülasyonun görselleştirilmesi ve gerçek zamanlı klavye/fare etkileşimi (Human-in-the-loop testing) için.

3.1.3. Vektörel fizik motoru analizi ve geliştirme

Geliştirilecek yapay zeka ajanının doğru eğitilebilmesi, simülasyon ortamının fiziksel tutarlılığına bağlıdır. Mevcut kod tabanındaki Kartezyen koordinat sistemi analiz edilmiş ve öklid uzayında hareket denklemleri türetilmiştir. Özellikle dinamik engellerin ve ajanın çarpışma (collision) sonrası davranışlarını modellemek için "Elastik Çarpışma" prensipleri vektörel düzlemde kodlanmıştır. Bir engele çarpan ajanın hız vektörünün (v), yüzey normaline (n) göre yansıması (reflection) şu matematiksel modelle simülasyona entegre edilmiştir:

$$\mathrm{v}_{\mathrm{yeni}} = \mathrm{v}_{\mathrm{eski}} - 2(\mathrm{v}_{\mathrm{eski}} \cdot \mathrm{n})\mathrm{n}$$

Bu formülde $(\mathrm{v}_{\mathrm{eski}} \cdot \mathrm{n})$ iç çarpımı (dot product), hızın yüzeye dik bileşenini verir. Bu bileşenin 2 katının çıkarılması, vektörün yüzeye göre simetriğini (yansımasını) oluşturur. Ayrıca, enerji kaybını simüle etmek için bir sürtünme katsayısı ($k_{\mathrm{friction}} < 1$) eklenerek sistemin termodinamik gerçekçiliği artırılmıştır.

3.2. Yapay Zeka Modelinin Geliştirilmesi ve Optimizasyonu

Ajanın karar mekanizmasını oluşturan sinir ağları tasarlanmış ve öğrenme performansını artıracak derinlemesine optimizasyonlar gerçekleştirilmiştir.

3.2.1. DQN algoritması ve matematiksel modeli

Ajanın eğitimi için temel olarak "Model-Free" ve "Off-Policy" bir yapı olan Derin Q-Ağı (DQN) algoritması kullanılmıştır. Ancak standart DQN yapısının bazı kronik sorunlarını (aşırı tahmin sapması vb.) gidermek amacıyla, literatürde kabul görmüş Double DQN ve Dueling Network mimarileri de incelenmiş ve entegre edilmiştir.

Standart DQN ve Bellman Denklemi

DQN algoritmasında amaç, Bellman Optimallik Denklemi ile tanımlanan Q fonksiyonunu bir sinir ağı $Q(s, a; \theta)$ ile yakınsamaktır. Eğitim sürecinde minimize edilen Kayıp (Loss) Fonksiyonu şu şekilde kurgulanmıştır

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(r + \gamma\, max_{a'}\, Q(s', a'; \theta^{-}) - Q(s, a; \theta)\right)^2\right] [6]$$

Burada θ ana ağın parametrelerini, θ^{-} ise hedef ağın (target network) parametrelerini temsil eder.

Double DQN (DDQN) ve Aşırı Tahmin (Overestimation) Sorunu

Standart DQN algoritması, hedef Q değerini hesaplarken max operatörünü kullandığı için Q değerlerini olduğundan yüksek tahmin etme (Overestimation Bias) eğilimindedir. Bu durum, ajanın suboptimal (en iyi olmayan) aksiyonlara aşırı güvenmesine neden olur. Bu sorunu çözmek için tarafından önerilen Double DQN mimarisi kullanılmıştır. Bu yöntemde aksiyonun seçimi ile aksiyonun değerlemesi birbirinden ayrılır:

1. En iyi aksiyon, güncel ana ağ (θ) kullanılarak seçilir.

2. Bu aksiyonun Q değeri, hedef ağ (θ^-) kullanılarak hesaplanır.

Double DQN için güncellenmiş hedef fonksiyonu şöyledir:

$$Y_{DoubleDQN} = r + \gamma\, Q(s', argmax_{a'}\, Q(s', a'; \theta);\ \theta^-)$$

Dueling DQN Mimarisinin Entegrasyonu

Standart DQN mimarisinde, sinir ağı her durum-aksiyon çifti için tek bir Q değeri üretir. Ancak bazı durumlarda (örneğin önünde engel olmayan bir yolda giden araç için), hangi aksiyonun seçildiğinden ziyade, o anki durumun ne kadar "iyi" olduğu (State Value) daha önemlidir. Bu ayrımı yapabilmek için Dueling Network mimarisi projeye dahil edilmiştir.

Bu mimaride, sinir ağının son katmanları iki ayrı kola ayrılır:

1. *Değer Akışı (Value Stream - $V(s)$):* O anki durumun ne kadar değerli olduğunu skaler olarak tahmin eder.

2. *Avantaj Akışı (Advantage Stream - $A(s,a)$):* Her bir aksiyonun, diğerlerine göre ne kadar avantajlı olduğunu hesaplar.

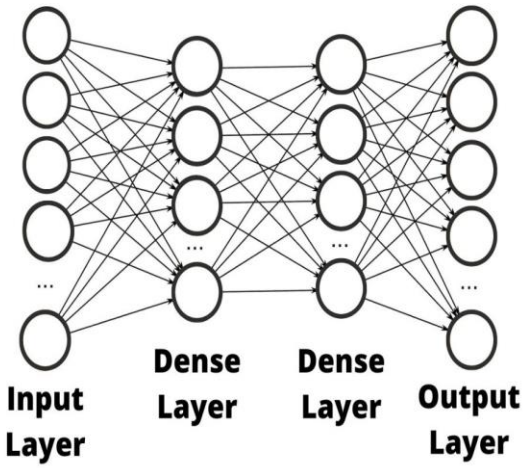
Bu iki akış, çıkış katmanında şu formülle birleştirilerek nihai Q değerini oluşturur:

$$Q(s,a) = V(s) + (A(s,a) - \frac{1}{|A|}\sum_{a'} A(s,a'))$$

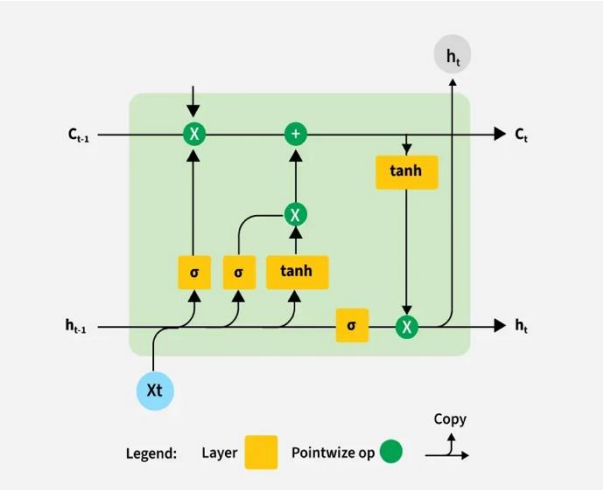
Dueling mimarisi sayesinde ajan, her aksiyonun sonucunu tek tek öğrenmek zorunda kalmadan, hangi durumların (states) değerli olduğunu daha hızlı öğrenmiş ve eğitim süresi kısalmıştır.

3.2.2. Ağ mimarisi karşılaştırmaları: lstm ve dense layer

Eğitim sürecinde, ajanın geçmiş durumları (Trajectory) hatırlayarak, engellerin hareket yönünü kestirebilmesi hipoteziyle LSTM (Long Short-Term Memory) tabanlı rekürsif bir ağ mimarisi denenmiştir. LSTM hücresinin içindeki "Unutma Kapısı" (Forget Gate) ve "Giriş Kapısı" (Input Gate) mekanizmaları teorik olarak avantajlı görünse de; yapılan deneysel testlerde, LSTM katmanlarının geriye yayılım (BPTT - Backpropagation Through Time) maliyeti nedeniyle eğitim süresini %60 oranında artırdığı tespit edilmiştir. Buna karşılık başarı oranında (Capture Rate) sadece %2'lik marjinal bir katkı sağladığı görülmüştür. Maliyet/Performans analizi sonucunda, daha hafif ve çıkarım (inference) süresi daha hızlı olan Dense (Tam Bağlantılı) mimaride karar kılınmıştır.



Şekil3.1: Dense Layer



Şekil 3.2: LSTM

3.2.3. Hiperparametre Optimizasyonu

Modelin stabilitesi için şu kritik parametreler optimize edilmiştir:

Öğrenme Oranı (Learning Rate - α): Gradyan inişi (Gradient Descent) sırasındaki adım büyüklüğünü ifade eden öğrenme oranı α), eğitimin başarısını doğrudan etkileyen en kritik parametredir. Sabit bir değer kullanıldığında; çok yüksek değerlerin ($> 10^{-3}$) optimizasyon sürecinde osilasyona (kararsız salınımlara), çok düşük değerlerin ($< 10^{-6}$)ise modelin yerel minimuma (local minima) takılmasına ve eğitimin çok yavaşlamasına neden olduğu gözlemlenmiştir.

Bu ikilemi çözmek için sabit bir oran yerine Learning Rate Scheduler (Öğrenme Oranı Planlayıcısı) stratejisi entegre edilmiştir.

Bu strateji ile eğitim sürecinin başında daha yüksek bir oran (10^{-4}) kullanılarak kayıp (loss) yüzeyinde hızlı ilerleme sağlanmış, eğitim ilerledikçe bu oran dinamik olarak (örneğin lineer veya üstel azalarak) (10^{-5}) seviyelerine düşürülmüştür. Böylece modelin eğitimin son aşamalarında "fine-tuning" (ince ayar) yaparak en optimum noktaya hassas bir şekilde yakınsaması sağlanmıştır.

İndirim Faktörü (Gamma - γ): Pekiştirmeli öğrenme problemlerinde ajanın amacı, anlık ödülü değil, gelecekte toplayacağı toplam ödülü (G_t) maksimize etmektir. Matematiksel olarak bu toplam, indirim faktörü γ ile şu şekilde ifade edilir:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^{\mathrm{k}} \mathrm{R_t} + \mathrm{k} + 1 \quad [4]$$

Burada γ parametresi ($0 < \gamma < 1$) ödüllerin değerinin zamanla ne kadar azalacağını belirler.

- *Düşük γ (örn: 0,1):* Ajan "miyopik" (myopic) davranır; sadece anlık ödüllere odaklanır, geleceği önemsemez.

- *Yüksek γ (örn: 0,99):* Ajan "ileri görüşlü" davranır; şu anki küçük cezaları (örneğin engelin etrafından dolaşmak için yol uzatmak), gelecekteki büyük ödül (hedefi yakalamak) için göze alır.

Takip-Kaçış senaryosu, doğası gereği "Gecikmiş Ödül" (Delayed Reward) yapısına sahiptir; yani ajanın yaptığı doğru manevraların asıl ödülü (yakalama), yüzlerce adım sonra gelmektedir. Bu nedenle projede $\gamma = 0{,}99$ seçilmiştir. Bu değer, ajanın efektif zaman ufkunu (Effective Time Horizon $\approx 1/(1-\gamma)$ yaklaşık 100 adım olarak belirler. Bu sayede ajan, sadece önündeki engeli değil, 100 adım sonrasındaki olası hedef pozisyonunu da hesaba katarak stratejik rotalar oluşturabilmiştir.

3.2.4. Stabilite Çalışmaları

Standart DQN eğitim süreçlerinde, ajanın yeni stratejiler öğrenirken daha önce edindiği kritik deneyimleri hafızasından silmesi veya üzerine yazması durumu olan "katastrofik unutma" (catastrophic forgetting) sorunu ile karşılaşılmıştır. Bu durum, eğitim grafiklerinde ani performans düşüşlerine ve kararsızlığa yol açmıştır. Bu problemi çözmek ve ajanın keşif-sömürü (exploration-exploitation) dengesini korumak adına, öncelikle ε (Epsilon) decay stratejisi lineer bir yapıya dönüştürülerek yeniden yapılandırılmıştır. Ancak tek başına parametre optimizasyonunun yeterli gelmemesi üzerine, algoritma mimarisine iki kritik yapısal eklenti yapılmıştır: Ajanın hatalarından daha fazla ders çıkarmasını sağlayan Öncelikli Deneyim Tekrarı (Prioritized Experience Replay - PER) mekanizması ve hedef ağ (Target Network) güncellemelerindeki osilasyonları engelleyen Yumuşak Güncelleme (Soft Update) tekniği sisteme entegre edilmiştir.

Linear Epsilon Decay: Pekiştirmeli öğrenmede ajanın rastgele hareket ederek ortamı tanıması (Keşif/Exploration) ile öğrendiği bilgiyi kullanarak en iyi ödülü toplaması (Sömürü/Exploitation) arasındaki denge, ε-Greedy (Epsilon-Açgözlü) stratejisi ile sağlanır. ε değeri, ajanın rastgele hareket etme olasılığını temsil eder (Başlangıçta ε =1,0).Standart uygulamalarda kullanılan Üstel Düşüş (Exponential Decay) yönteminin, keşif oranını çok hızlı düşürdüğü ve ajanın ortamı yeterince tanımadan "yerel optimuma" (local optimum) sıkışmasına neden olduğu gözlemlenmiştir. Bu projede, karmaşık harita yapısı nedeniyle daha uzun ve kararlı bir keşif süreci gerektiğinden Lineer Düşüş stratejisi benimsenmiştir. Epsilon değeri, eğitim adımı (t) boyunca şu formülle güncellenmiştir:

$$\epsilon_t = max(\epsilon_{minimum}, \epsilon_{start} - \frac{\epsilon_{start} - \epsilon_{minimum}}{N_{decay}} \cdot t) \text{ [6]}$$

Burada N_{decay}, keşfin tamamlanacağı toplam adım sayısını ifade eder. Bu strateji sayesinde, ajanın eğitim sürecinin ilk %35'lik diliminde ortamın tüm köşelerini (corner cases) deneyimlemesi garanti altına alınmış ve erken yakınsama (premature convergence) sorunu çözülmüştür.

Polyak Averaging (Soft Update): Hedef ağ (Target Network) parametrelerinin kopyalanması sırasında ani değişimleri önlemek için "Yumuşak Güncelleme" stratejisi uygulanmıştır. Her adımda hedef ağ ağırlıkları θ' şu formülle güncellenir:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta' \text{ [7]}$$

Burada τ , 0,005 gibi çok küçük bir katsayıdır. Bu teknik, eğitim grafiğindeki ani sıçramaları (spikes) engellemiştir.

Prioritized Experience Replay (PER): Standart yöntemde deneyimler hafızadan rastgele (Uniform Distribution) çekilir yaklaşımı ile, ajanın "şaşırdığı" (TD-Hatası yüksek olan) deneyimlere öncelik verilmiştir. Bir deneyimin seçilme olasılığı $P(i)$ şu şekilde formüle edilmiştir:

$$P(i) = \frac{|\delta_i|^\alpha}{\sum_k |\delta_k|^\alpha} \text{ [13]}$$

Burada δ_i beklenen değer ile gerçekleşen değer arasındaki farktır.

Bu iyileştirmeler sonucunda ajanın başarı oranı (capture rate) %60 seviyesinden %70 seviyesine yükseltilmiştir.

3.3. Ödül Fonksiyonu Mühendisliği ve Ortam İyileştirmeleri

Pekiştirmeli Öğrenme problemlerinde, ajanın davranışını şekillendiren en kritik bileşen ödül fonksiyonudur (Reward Function). Takip-Kaçış gibi karmaşık senaryolarda, ajanın hedefe ulaşana kadar hiçbir geri bildirim almaması durumu olan "Seyrek Ödül" (Sparse Reward) problemi, eğitimin yakınsamamasına veya çok yavaş ilerlemesine neden olmaktadır. Bu problemi aşmak ve öğrenme sürecindeki gradyan akışını (gradient flow) sürekli kılmak amacıyla, literatürde "Potential-Based Reward Shaping" olarak bilinen yöntem uygulanmıştır. Bu yaklaşımda ödül mekanizması; sadece sonuç odaklı (Terminal) değil, süreci iyileştiren (Shaping) bileşenleri de içerecek şekilde çok katmanlı bir yapıya dönüştürülmüştür.

Kurgulanan ödül fonksiyonu R_t Seyrüsefer (Navigation), Güvenlik (Safety) ve Görev (Task) olmak üzere üç ana kategoride toplanan şu bileşenlerden oluşturulmuştur:

$$R_t = \underbrace{R_{mesafe} + R_{zaman}}_{\text{Seyrüsefer}} + \underbrace{R_{carpma} + R_{radar}}_{\text{Güvenlik}} + \underbrace{R_{hedef}}_{\text{Görev}}$$

Seyrüsefer (Navigation): Ajanın en kısa rotayı bulmasını teşvik etmek için; hedefe yaklaşılan her adımda pozitif ödül (R_{mesafe}) geçen her simülasyon adımında ise sabit bir "zaman cezası" (R_{zaman}) uygulanarak ajanın tembel davranması engellenmiştir.

Güvenlik ve Taktik (Safety & Stealth): Ajanın fiziksel bütünlüğünü ve operasyonel gizliliğini korumayı öğrenmesi adına; statik/dinamik engellere çarpma ve takım arkadaşlarıyla çarpışma (R_{carpma}) ve düşman radar menziline girme (R_{radar}) durumlarına yüksek cezalar atanmıştır.

Terminal Ödül (Terminal): Hedef yakalandığında verilen büyük pozitif ödül (R_{hedef}) ile epizot sonlandırılarak ağın nihai amacı öğrenmesi sağlanmıştır.

*Sonuç ve Davranışsal Analiz:*Ödül fonksiyonu mimarisinde gerçekleştirilen bu çok katmanlı (hierarchical) düzenlemeler ve simülasyon ortamındaki başlangıç (spawn) algoritmalarındaki hataların giderilmesi, eğitim sonuçlarında belirgin bir iyileşme sağlamıştır. Özellikle "şekillendirici" (shaping) ödüllerin entegrasyonu sonrasında, ajanın davranış modellerinde şu kritik değişimler gözlemlenmiştir:

Taktiksel Farkındalık: Ajan, sadece en kısa yolu (shortest path) hesaplamak yerine; düşman radarlarının kör noktalarını kullanan ve takım arkadaşlarıyla çarpışma riskini minimize eden güvenli rotaları tercih etmeyi öğrenmiştir.

Kararlılık: Başlangıçta sıkça görülen kendi etrafında dönme veya engellere takılma gibi kararsız davranışlar (oscillations), yerini hedefe kilitlenen akıcı manevralara bırakmıştır.

Performans Artışı: Nicel veriler ışığında, yapılan bu iyileştirmelerin modelin genel başarısına doğrudan etki ettiği kanıtlanmıştır. Eğitim öncesi (Baseline) testlerde %70 bandında seyreden Yakalama Oranı (Capture Rate), optimizasyonlar sonrasında istikrarlı bir şekilde %90 seviyesine yükseltilmiştir.

3.4. Test ve Doğrulama Altyapısı: İnteraktif Senaryo Oluşturucu (GUI)

Derin Pekiştirmeli Öğrenme modellerinin eğitimi genellikle rastgele oluşturulan (randomized) ortamlarda gerçekleştirilir. Ancak modelin başarısını tam olarak ölçmek ve uç durumlardaki (corner cases) davranışlarını analiz edebilmek için, kontrol edilebilir ve tekrarlanabilir test senaryolarına ihtiyaç duyulmaktadır. Bu ihtiyacı karşılamak amacıyla, eğitim döngüsünden bağımsız çalışan, Pygame kütüphanesi üzerine inşa edilmiş özgün bir "İnteraktif Senaryo Oluşturucu ve Test Arayüzü" tasarlanıp geliştirilmiştir.

Geliştirilen bu yazılım, projenin "Sim-to-Real" geçiş aşamasında bir köprü görevi görerek, simülasyon ortamındaki parametrelerin görsel olarak yönetilmesine olanak tanımaktadır. Araç, "Tasarım Modu" (Editor Mode) ve "Simülasyon Modu" (Run Mode) olmak üzere iki temel mimari üzerine kurgulanmıştır.

3.4.1. Tasarım modu ve dinamik çevre kurgusu

Arayüzün tasarım modu, kullanıcının fare ve klavye etkileşimleri ile simülasyon dünyasını sıfırdan inşa etmesine olanak tanır. Kullanıcı, arayüz üzerindeki araç çubuğunu kullanarak şu bileşenleri sahneye yerleştirebilmektedir:

Ajan ve Hedef Konumlandırma: Takipçi (Pursuer) ve Kaçan (Evader) ajanların başlangıç koordinatları (spawn points) ve başlangıç yönelim açıları (heading), sürükle-bırak yöntemiyle mikroskobik hassasiyette ayarlanabilir.

Statik ve Dinamik Engeller: Harita üzerine farklı geometrilere sahip duvarlar ve engeller eklenebilir. Bu sayede "L" biçimli koridorlar, dar geçitler veya kapalı odalar gibi ajanı zorlayacak topolojiler manuel olarak oluşturulabilir.

Güvenli Bölge Tanımlama: Senaryo gereği hedefin ulaşmaya çalıştığı güvenli bölgeler (Safe Zones) görsel olarak işaretlenebilir.

Tasarım aşamasında, yerleştirilen nesnelerin fiziksel çakışmaları (collision overlap) gerçek zamanlı olarak hesaplanarak, kullanıcının hatalı bir yerleşim yapması (örneğin bir ajanı duvarın içine koyması) engellenmiştir.

3.4.2. Veri kalıcılığı ve JSON serileştirme

Oluşturulan karmaşık senaryoların kaybolmaması ve farklı algoritmaların (DQN vs PPO) aynı senaryo üzerinde adil bir şekilde karşılaştırılabilmesi için bir veri kayıt sistemi geliştirilmiştir. Tasarlanan sahneler, JSON (JavaScript Object Notation) formatında serileştirilerek dosya sistemine kaydedilmektedir. Bu yapı sayesinde; nesnelerin koordinatları, türleri, hız vektörleri ve fiziksel özellikleri yapılandırılmış bir veri formatında saklanır. Kaydedilen senaryolar, daha sonra tek bir komutla tekrar yüklenerek (Deserialization), testlerin tekrarlanabilirliği (reproducibility) garanti altına alınmıştır. Bu özellik, bilimsel raporlama sürecinde aynı test koşullarının sağlanması açısından kritik bir rol oynamıştır.

3.4.3. Hibrit Kontrol Mekanizması

Aracın en önemli yeteneklerinden biri, "Otonom" ve "Manuel" kontrol modları arasında gerçek zamanlı geçiş yapabilmesidir.

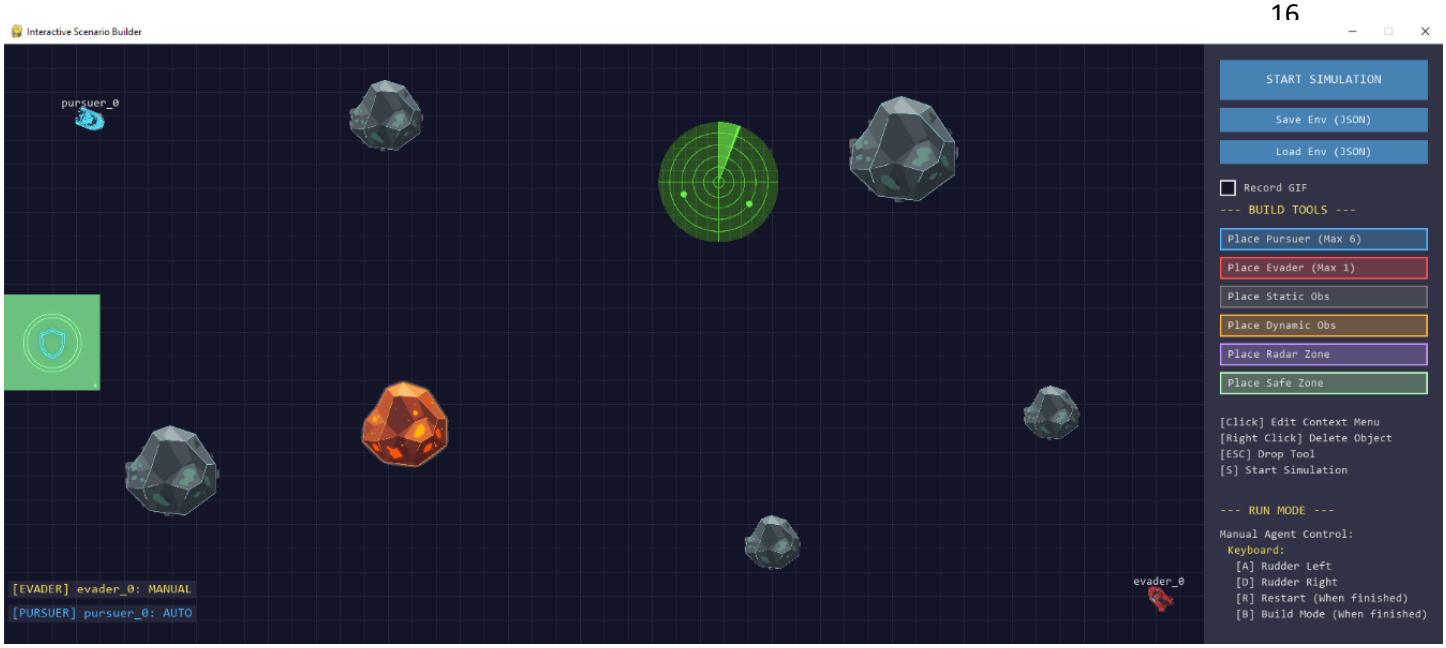
Otonom Mod: Ajan, eğitilmiş sinir ağından (Inference Model) gelen komutlarla hareket eder. Bu modda modelin başarısı gözlemlenir.

Manuel Mod: Kullanıcı, klavye girdileriyle ajanın kontrolünü anlık olarak devralabilir. Bu özellik, ajanın belirli bir durumda sıkışması halinde müdahale etmek veya ajana "nasıl davranması gerektiğini" göstermek (Demonstration) amacıyla kullanılmıştır. Bu yapı, literatürde "Human-in-the-Loop" olarak bilinen test metodolojisinin uygulanmasını sağlamıştır.

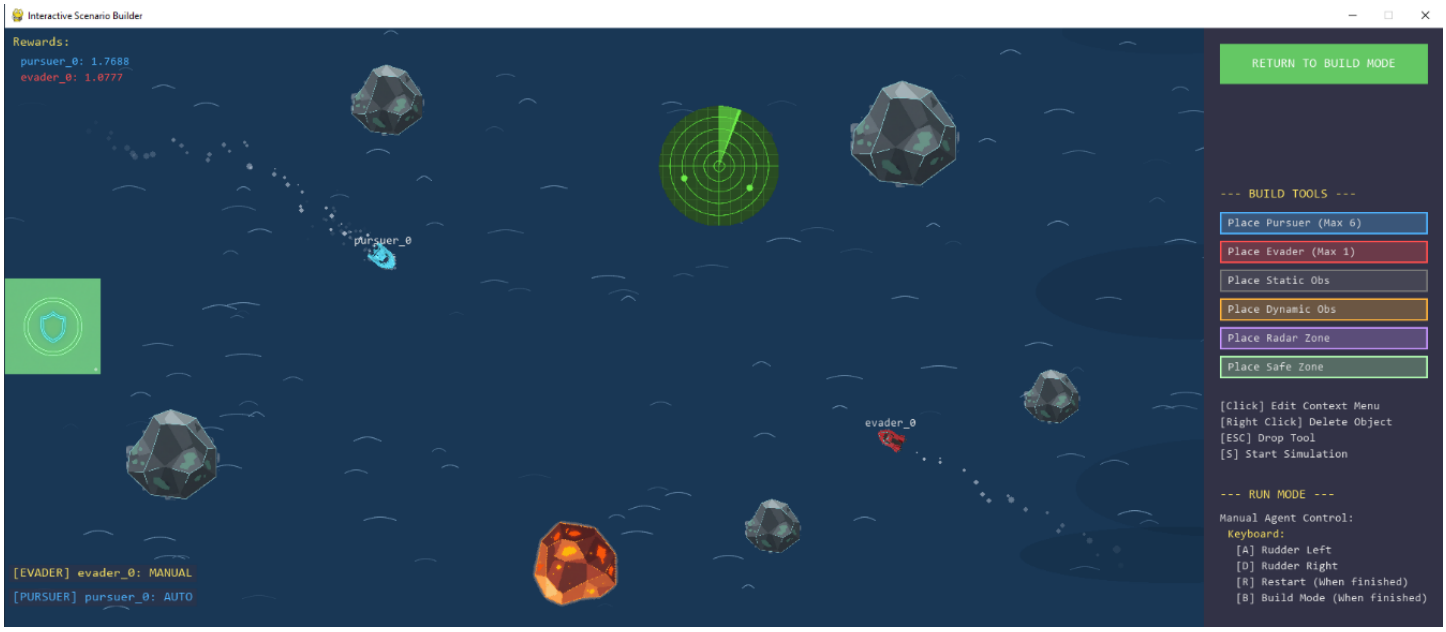
3.4.4. Görselleştirme ve Raporlama Modülü

Test sonuçlarının dokümante edilmesi amacıyla arayüze entegre bir kayıt modülü eklenmiştir. Simülasyonun her karesi (frame), arka planda çalışan bir tampon belleğe (buffer) alınarak, test sonunda otomatik olarak yüksek çözünürlüklü GIF veya video formatına dönüştürülmektedir. Bu özellik, ajanın manevra kabiliyetlerinin sunumlarda ve proje raporlarında görsel olarak kanıtlanmasını sağlamıştır.

Sonuç olarak; geliştirilen bu arayüz aracı, projenin sadece bir yazılım çıktısı olmaktan öte, modelin güvenilirliğini ve performansını doğrulayan kapsamlı bir mühendislik çalışması olduğunu ortaya koymaktadır.



Resim 3.1: İnteraktif Senaryo Oluşturucu Tasarım Modu



Resim 3.2: İnteraktif Senaryo Oluşturucu Çalıştırma Modu

3.4.5. Performans analizi

Geliştirilen interaktif test aracı kullanılarak modelin performansı; başarı oranı (capture rate), ortalama ödül ve yakalama süresi (step count) gibi temel metrikler üzerinden detaylıca analiz edilmiştir.

Bu analizler sırasında, simülasyon ortamının doğasında bulunan rastgeleliğin (stochasticity) sonuçlar üzerinde oluşturabileceği yanlılığı (bias) ortadan kaldırmak amacıyla deterministik bir test süreci kurgulanmıştır. Farklı eğitim modellerinin karşılaştırılmasında standart bir zemin oluşturmak adına "Random Seed" (rastgelelik çekirdeği) sabitleme mekanizması uygulanmıştır.

Bu teknik sayesinde, farklı test koşularında dahi aynı episode numaralarında tıpatıp aynı çevre düzeninin (statik/dinamik engel konumları, ajan başlangıç noktaları vb.) oluşması garanti altına alınmıştır. Yapılan bu "seed" ayarlaması ile performans farklarının şans faktöründen değil, algoritmaların öğrenme kalitesinden kaynaklandığı netleştirilmiş ve modelin davranış tutarlılığı bilimsel yöntemlerle kanıtlanmıştır.

3.5. Donanım Uyumluluğu ve Gerçek Zamanlılık Çalışmaları

Projenin uygulama aşamasında, geliştirilen simülasyon ortamının gerçek donanım (Real-Time Hardware) kısıtlarına ve gömülü sistemlerin çalışma frekanslarına uyumlu hale getirilmesi amacıyla kapsamlı optimizasyon çalışmaları yürütülmüştür.

3.5.1. Simülasyon zaman adımı (dt) uyarlaması

Gerçek dünyadaki İDA otopilot donanımlarının kontrol döngüsü frekansları (genellikle 10Hz - 20Hz) referans alınarak, simülasyonun fiziksel zaman adımı (dt) 0.1 saniye olarak revize edilmiştir. Bu ayarlama, simülasyonun gerçek zamanlı (Real-Time) çalışmasına olanak tanısa da; ajanın çevresel değişimleri "kesik kesik" (discontinuous) algılamasına yol açmış ve karar verme mekanizmasında tepki süresi (reaction time) sorunları yaratarak öğrenme performansını düşürmüştür.

3.5.2. Frame skipping (kare atlama) yöntemi

Düşük dt değerinden kaynaklanan bu algısal süreksizlik sorununu çözmek ve işlem maliyetini düşürmek için literatürde sıkça başvurulan "Frame Skipping" tekniği uygulanmıştır. Bu yöntemde ajan, simülasyonun her adımında değil, her k adımda bir (örneğin k=5) karar verir ve seçtiği aksiyonu aradaki adımlar boyunca tekrar eder.

Bu teknik sayesinde , hareket sürekliliği sağlanmıştır, Ajanın mikroskobik titremeler yerine daha kararlı ve uzun vadeli manevralar yapması sağlanarak, öğrenme performansı hedeflenen seviyeye tekrar yükseltilmiştir.

3.6. Fizik Motoru Değişimi ve İleri Simülasyon Altyapısı (MuJoCo)

Projenin ilk aşamalarında kullanılan 2D Pygame ortamı, algoritmaların temel mantığının doğrulanmasında başarılı olmuştur. Ancak, gerçek dünya dinamiklerinin (sürtünme, atalet, temas kuvvetleri) hassas modellenebilmesi ve endüstriyel standartların yakalanması amacıyla, stajın son bölümünde MuJoCo (Multi-Joint dynamics with Contact) fizik motoruna ve Stable Baselines3 kütüphanesine geçiş yapılmıştır.

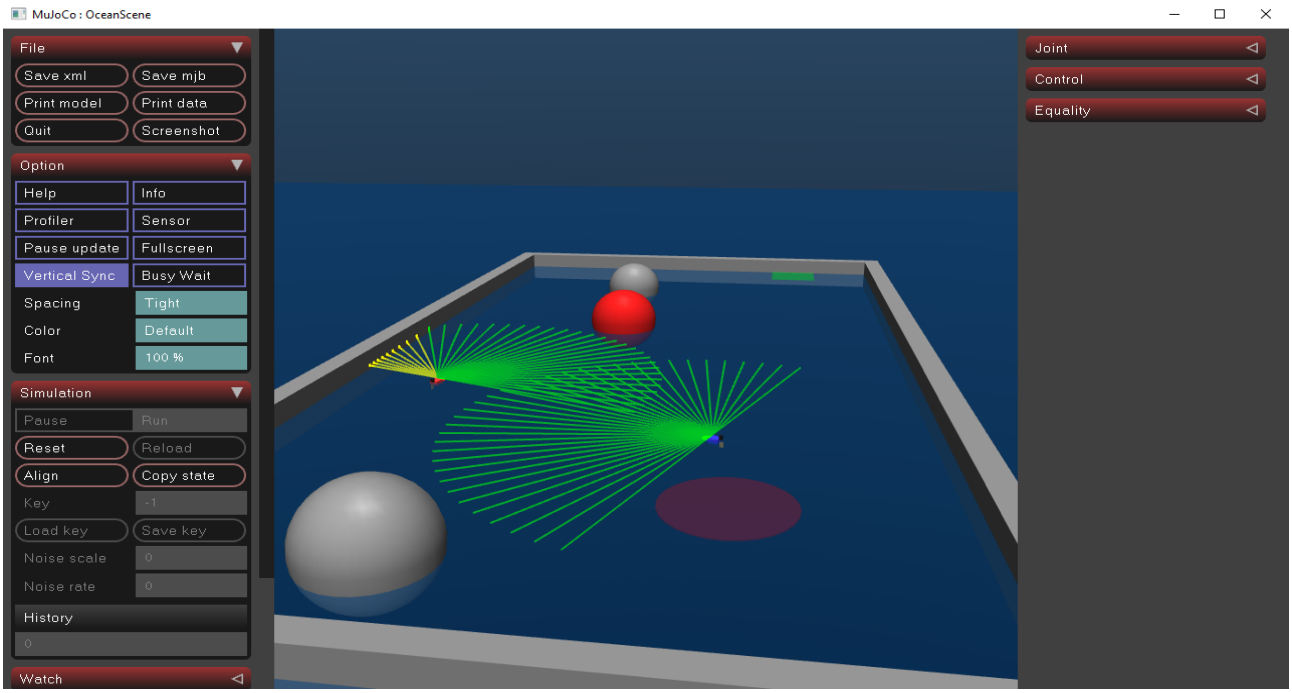
3.6.1. Hidrodinamik modelleme ve hareket denklemleri

MuJoCo varsayılan olarak katı cisim dinamiği üzerine kurulu olduğundan, suyun etkileşimlerini simüle etmek için özel eklentiler geliştirilmiştir. İDA gövdesinin su altında kalan hacmi üzerinden Arşimet prensibine göre kaldırma kuvveti (buoyancy) dinamik olarak hesaplanmıştır. Aracın hareket karakteristiği ise denizcilik literatüründe standart kabul edilen Thor I. Fossen'in matematiksel modelleri referans alınarak kurgulanmıştır.

Aracın 6 serbestlik derecesindeki (6-DOF) hareketi, hidrodinamik sönümleme ($D(v)$), eklenmiş kütle (M) ve Coriolis ($C(v)$) etkilerini içeren şu denklemle modellenmiştir:

$$Mv + C(v)v + D(v)v + g(\eta) = \tau$$

Bu modelleme sayesinde geliştirilen ajan, sadece hedefi takip etmeyi değil; suyun direncini ve aracın ataletini de hesaba katarak optimum manevrayı yapmayı öğrenmiştir.



Resim 3.3: Mujoco Deniz Ortamı

3.6.2. Stable Baselines3 entegrasyonu ve PPO algoritması

Projenin MuJoCo tabanlı fizik motoruna taşınmasıyla birlikte, geliştirme sürecinin hızlandırılması ve algoritmaların endüstriyel standartlarda karşılaştırılabilmesi (Benchmarking) amacıyla Stable Baselines3 (SB3) kütüphanesi projeye entegre edilmiştir. Önceki aşamalarda manuel olarak (from scratch) yazılan PyTorch eğitim döngüleri, öğrenme amacıyla oldukça faydalı olsa da; karmaşık 3D fizik ortamlarında hiperparametre yönetimi, paralel eğitim ve loglama süreçlerinde yetersiz kalmaya başlamıştır. SB3 entegrasyonu, bu süreçleri modüler bir yapıya kavuşturmuştur.

a) Sürekli Aksiyon Uzayı ve PPO Tercihi

Projenin ilk fazında kullanılan DQN algoritması, yapısı gereği ayrık aksiyon uzaylarında (Discrete Action Space) çalışmaktadır. Ancak gerçek dünyadaki bir İDA'nın gaz kolu (Throttle) veya dümen açısı (Rudder Angle) 0 ile 1 arasında sonsuz değer alabilen sürekli değişkenlerdir. DQN'i bu yapıya uyarlamak için yapılan "ayrıklaştırma" (discretization) işlemi, robotun hareketlerinde keskinliklere ve titreşimlere yol açmıştır.

Bu sorunu çözmek için, Proksimal Politika Optimizasyonu (Proximal Policy Optimization - PPO) algoritmasına geçiş yapılmıştır. PPO, bir "Actor-Critic" mimarisi kullanarak doğrudan sürekli değerler (örneğin: gaz=%42.5, dümen=-12.3°) üretir. Algoritmanın kalbinde yer alan "Clipped Surrogate Objective" fonksiyonu, eğitimin kararlılığı için kritik bir rol oynamıştır. Standart Gradyan İnişi yöntemlerinde, çok büyük bir güncelleme adımı politikanın tamamen bozulmasına (policy collapse) neden olabilirken; PPO, güncelleme oranını belirli bir aralıkta (genellikle $\varepsilon = 0{,}2$) "kırparak" (clipping) güvenli bir öğrenme süreci sağlar. Matematiksel olarak optimize edilen hedef fonksiyon şöyledir:

$$L^{CLIP}(\theta) = \mathbb{E}_t\left[min\left(r_{t(\theta)}\,\widehat{\mathrm{A}}_{\mathrm{t}}, \mathrm{clip}\left(r_{t(\backslash theta)}, 1-\epsilon, 1+\epsilon\right)\widehat{\mathrm{A}}_{\mathrm{t}}\right)\right][8]$$

Bu koruma mekanizması sayesinde, MuJoCo ortamındaki karmaşık hidrodinamik kuvvetlere (sürtünme, kaldırma) rağmen, ajanın öğrenme süreci osilasyona girmeden stabil bir şekilde ilerlemiştir.

b) Paralel Eğitim Mimarisi (Vectorized Environments)

Derin öğrenme modellerinin eğitimindeki en büyük darboğaz, simülasyon ortamından veri toplama hızıdır. SB3 kütüphanesinin sağladığı VectorEnv sınıfı ve SubprocVecEnv alt yapısı kullanılarak, donanımın çok çekirdekli (Multi-Core) mimarisinden tam verim alınmıştır. Tek bir simülasyon ortamı yerine, işlemci çekirdeklerine dağıtılmış 8 farklı simülasyon örneği (instance) paralel olarak koşturulmuştur.

Bu yöntemle, ajanın deneyim havuzuna (Rollout Buffer) veri akışı 8 kat hızlandırılmış, böylece normalde günler sürecek eğitimler saatler mertebesine indirilmiştir. Ajanlar farklı "Random Seed" değerleriyle başlatılarak, ortamın farklı varyasyonlarını aynı anda deneyimlemiş ve modelin genelleme (generalization) yeteneği artırılmıştır.

c) Denetim ve İzleme Mekanizmaları (Custom Callbacks)

Eğitim sürecini "Siyah Kutu" olmaktan çıkarmak ve modele müdahale edebilmek için SB3'ün BaseCallback sınıfından türetilen özel denetim mekanizmaları geliştirilmiştir:

EvalCallback: Eğitim devam ederken, belirli aralıklarla (örn: her 10.000 adımda) modelin güncel kopyası ayrı bir test ortamında değerlendirilmiştir. Eğer model yeni bir "En Yüksek Skor" (Best Mean Reward) elde ederse, ağırlıkları `.zip` formatında otomatik olarak kaydedilmiştir. Bu sayede "Overfitting" (Aşırı Öğrenme) başladığı anda en iyi modelin kaybedilmesi engellenmiştir.

CheckpointCallback: Elektrik kesintisi veya sistem hatası gibi durumlara karşı, modelin belirli periyotlarla yedeklenmesi sağlanmıştır.

Tensorboard Entegrasyonu: Callback yapısı üzerinden; anlık ödül, kayıp (loss) değeri, FPS ve öğrenme oranı gibi metrikler gerçek zamanlı olarak loglanmış ve grafiksel olarak analiz edilmiştir.

Sonuç: Stable Baselines3 entegrasyonu, projenin sadece akademik bir çalışma olmaktan çıkıp, sürdürülebilir ve ölçeklenebilir bir Ar-Ge projesine dönüşmesini sağlamıştır. PPO algoritmasının sürekli kontrol yeteneği ve VectorEnv mimarisinin sağladığı veri işleme hızı, projenin nihai hedefi olan "Sim-to-Real" vizyonu için gerekli olan sağlam altyapıyı oluşturmuştur.

3.6.3. Genişletilebilir modüler mimari: kara, hava ve deniz Entegrasyonu

Geliştirilen simülasyon altyapısı, sadece mevcut deniz ortamı (İnsansız Deniz Aracı - İDA) ile sınırlı kalmayacak şekilde, "Domain Agnostic" (Ortamdan Bağımsız) bir prensiple tasarlanmıştır.

Yazılım mimarisi, farklı platformların kolayca entegre edilebilmesi için şu şekilde kurgulanmıştır:

Soyut Sınıf Yapısı (Abstract Base Classes): Ajanın hareket dinamikleri ve sensör verileri soyutlanmıştır. Şu an aktif olan "Deniz Modülü"ne ek olarak, ileride "Kara Modülü" (tekerlek sürtünmesi ve arazi eğimi) veya "Hava Modülü" (aerodinamik ve irtifa kontrolü) eklendiğinde, ana RL algoritmasında (PPO/DQN/A2C) herhangi bir değişiklik yapılmasına gerek kalmayacaktır.

Varlık (Asset) Yönetimi: Simülasyon ortamı, XML tabanlı varlık dosyalarını modüler olarak yükleyecek şekilde yapılandırılmıştır. Bu sayede, sadece model dosyası değiştirilerek bir İDA simülasyonu, saniyeler içinde engebeli arazide giden bir İKA (İnsansız Kara Aracı) veya engellerin arasından süzülen bir İHA (İnsansız Hava Aracı) simülasyonuna dönüştürülebilir.

4. PROJE KONUSU : DERİN PEKİŞTİRMELİ ÖĞRENME İLE OTONOM TAKİP-KAÇIŞ SİSTEMİNİN GELİŞTİRİLMESİ

4.1. Giriş

Günümüz savunma sanayii ve robotik uygulamalarında, insansız sistemlerin otonomi seviyelerinin artırılması kritik bir ihtiyaç haline gelmiştir. Özellikle İnsansız Deniz Araçlarının (İDA), operatör müdahalesi olmaksızın karmaşık ve değişken çevre koşullarında görev yapabilmesi, "Tam Otonom" (Level 5 Autonomy) sistemlerin nihai hedefidir. Bu proje kapsamında ele alınan Takip-Kaçış (Pursuit-Evasion) problemi, bir veya birden fazla takipçi (pursuer) ajanın, kaçan bir hedefi (evader) engellerle dolu bir ortamda yakalamaya çalıştığı, dinamik ve stokastik (rastgele) bileşenler içeren bir optimizasyon problemidir [10, 12].

Literatür incelendiğinde, bu problemin çözümünde A* veya RRT (Rapidly-exploring Random Tree) gibi klasik yol planlama algoritmalarının, ortamın statik olduğu varsayımına dayandığı ve dinamik engeller karşısında yetersiz kaldığı görülmektedir. Oyun Teorisi tabanlı yaklaşımlar ise (Minimax vb.) durum uzayı büyüdüğünde hesaplama maliyeti açısından uygulanabilirliğini yitirmektedir. Bu kısıtlar ışığında, projenin çözüm mimarisinde Derin Pekiştirmeli Öğrenme (Deep Reinforcement Learning - DRL) yöntemleri tercih edilmiştir [4, 6].

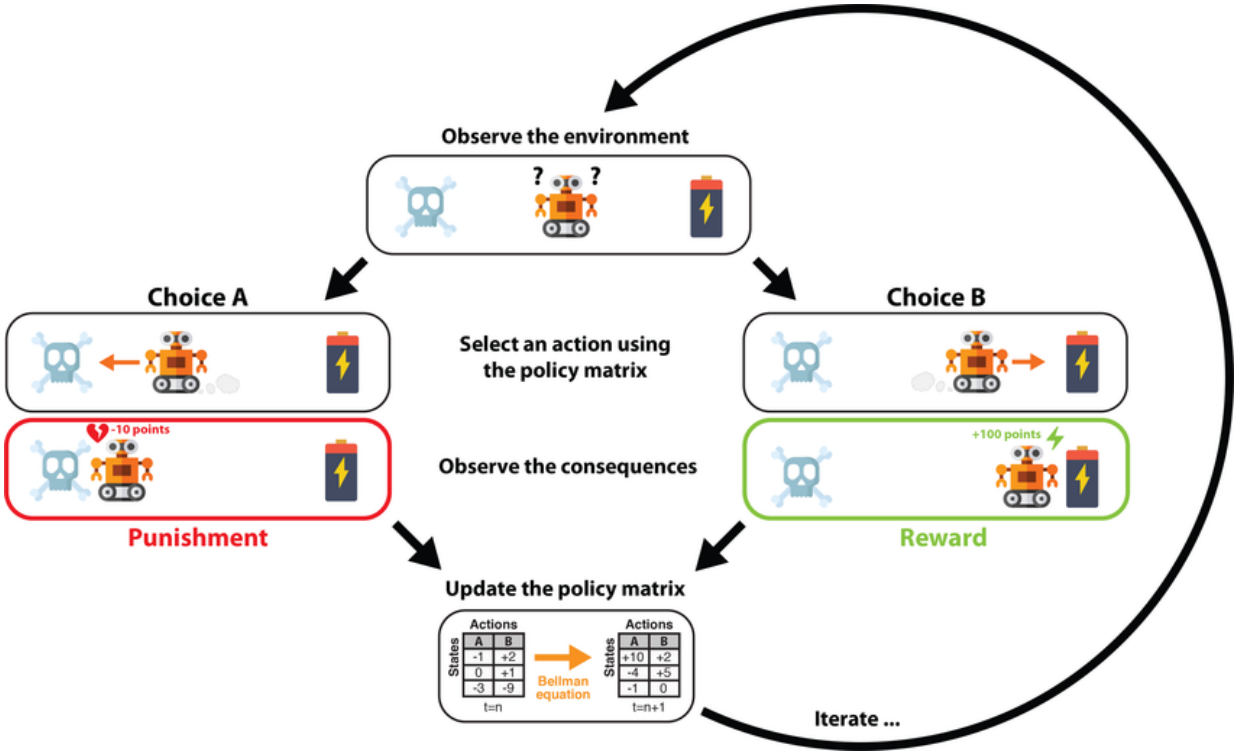
Projenin temel amacı; ajanın ham sensör verilerini işleyerek, "uçtan uca" (end-to-end) bir öğrenme süreciyle optimum manevra stratejilerini geliştirmesidir. Çalışmanın özgün değeri; literatürdeki standart ızgara (grid) tabanlı yaklaşımların aksine, MuJoCo fizik motoru kullanılarak hidrodinamik kuvvetlerin (sürtünme, atalet, Coriolis etkisi) modellendiği gerçekçi bir ortamda, Dueling DQN ve PPO gibi modern algoritmaların hibrit bir yapıda optimize edilmesidir [8, 15].

4.2. Yöntem ve Teknikler

Projenin gerçekleştirilmesinde, problemin matematiksel olarak modellenmesi (MDP), sistem mimarisinin kurgulanması ve yapay zeka algoritmalarının entegrasyonu olmak üzere üç aşamalı bir yöntem izlenmiştir.

4.2.1. Problemin matematiksel modeli (POMDP)

Gerçek dünya senaryolarında bir otonom ajanın ortamın tamamını (God-Mode) bilmesi mümkün değildir. Ajanlar sadece sensörlerinin (Lidar, Radar vb.) izin verdiği ölçüde çevreyi algılayabilir. Bu nedenle, Takip-Kaçış problemi klasik Markov Karar Süreci (MDP) yerine, belirsizliğin ve eksik bilginin modellendiği Kısmi Gözlemlenebilir Markov Karar Süreci (Partially Observable Markov Decision Process - POMDP) olarak formüle edilmiştir [4].



Şekil 4.1: POMDP

Bir POMDP, 7 bileşenli bir demet $(S, A, T, R, \Omega, O, \gamma)$ ile tanımlanır. Burada Ω gözlem uzayını, O(o|s',a) ise ajanın s' durumunda o gözlemini yapma olasılığını ifade eder.

Gözlem Uzayı ve Öznitelik Vektörleri (Observation Vectors):

Ajanın sinir ağına beslenen girdi verisi (Input Layer), ham sensör verilerinin işlenip normalize edilmesiyle oluşturulan Öznitelik Vektörleri (Feature Vectors) üzerine kuruludur. Takipçi (Pursuer) ve Kaçan (Evader) ajanların görev tanımları farklı olduğundan, gözlem vektörleri de farklı yapılandırılmıştır.

1. Takipçi (Pursuer) Gözlem Vektörü (Toplam 76 Öznitelik):

Takipçi ajanın temel amacı koordinasyon ve yakalamadır. 64 boyutlu vektör şu bloklardan oluşur:

- *Öz Durum (Own State - 5):* $[x_{norm}, y_{norm}, v_x, v_y, aktiflik]$. Ajanın normalize edilmiş global konumu ve yerel hız vektörleri.

- *Takım Arkadaşları (Teammates - 25)*: Sürü koordinasyonu için diğer 5 takipçi ajanın bağıl konumu, hızı ve durum bilgisi: $[rel_x, rel_y, rel_{vx}, rel_{vy}, aktiflik] \times 5$.

- *Rakipler (Adversaries - 6):* Ortamdaki 2 kaçan hedefin tespit edilmesi için Polar Koordinatlar (Mesafe r ve Açı ϕ) kullanılarak hedefe yönelim kolaylaştırılmıştır: $[evader_r, evader_\phi, aktiflik] \times 2$.

- *Radar Algısı (Radar - 3):* Yasaklı bölgelerin veya tehdit unsurlarının bağıl konumu ve yarıçapı: $[rel_x, rel_y, radius]$.

- *LiDAR Sensörü (LiDAR - 36):* Statik engelleri ve duvarları algılamak için ajanın etrafına 180 derece yayılan 36 ışınlı sanal lazer tarayıcı verisi (mesafe).

- *Zaman (Time - 1):* Epizodun bitimine kalan süre (adım sayısı).

2. Kaçan (Evader) Gözlem Vektörü (Toplam 64 Öznitelik):

Kaçan ajanın amacı hayatta kalmak ve güvenli bölgeye ulaşmaktır. 56 boyutlu vektör şu bloklardan oluşur:

- *Öz Durum (Own State - 5):* $[x_{norm}, y_{norm}, v_x, v_y, aktiflik]$. Ajanın normalize edilmiş global konumu ve yerel hız vektörleri.

- *Takım Arkadaşı (Teammates - 1):* Senaryo gereği tek olan (1 adet) diğer kaçan ajanın bağıl verileri.

- *Tehdit Algısı (Adversaries - 18):* Peşindeki 6 takipçi ajanın anlık konum ve açı bilgilerini (Mesafe r ve Açı ϕ) sürekli izleyerek en boş kaçış rotasını hesaplar: $[pursuer_r, pursuer_\phi, aktiflik] \times 6$.

- Güvenli Bölgeler (Safe Zones - 3): Ulaşılması gereken hedef bölgenin bağıl koordinatları.

- LiDAR ve Zaman (37): 180 derece yayılan 36 ışınlı engel algılama sensörü ve zaman adımı.

b) Aksiyon Uzayı (A):

Ajanın sinir ağından çıkan kararların, simülasyon ortamındaki eyleyicilere (actuators) nasıl iletildiğini tanımlayan komut setidir. Kullanılan algoritmaya göre karar uzayı şu şekilde yapılandırılmıştır:

DQN Modeli (Ayrık Kontrol):

Standart DQN algoritması sürekli değerler üretemediği için, İDA'nın dümen açısı (δ) ayrıklaştırılarak (discretized) tanımlanmıştır [6]. Karar uzayı, maksimum dümen açısı olan $[-40°, +40°]$ aralığının $5°$ hassasiyetle bölünmesiyle oluşturulmuştur.

Bu işlem sonucunda toplam 17 adet ayrık aksiyon elde edilmiştir:

$$A_{DQN} = \{-40°, -35°, \dots, 0°, \dots, +35°, +40°\}$$

Burada negatif değerler sola dönüşü, $0°$ düz gitmeyi, pozitif değerler ise sağa dönüşü ifade eder. Ajan bu 17 açıdan birini seçerek rotasını belirler.

PPO Modeli (Sürekli Kontrol):

MuJoCo ortamında çalışan PPO algoritması için aksiyon uzayı sürekli (continuous) olarak tanımlanmıştır [8]. Ağ, [-1, 1] aralığında normalize edilmiş iki değer üretir:

1. *Gaz (Throttle):* Motor itki kuvvetini belirler.

2. *Dümen (Steering):* Dümen torkunu belirler.

   Bu yapı, ara değerlerin (örneğin $12.5°$ gibi) hassas bir şekilde uygulanmasına olanak tanır.

4.2.2. Sistem Mimarisi ve Karar Modülü

Geliştirilen otonom sistem mimarisi; Algı (Perception), Karar (Decision) ve Kontrol (Control) olmak üzere üç hiyerarşik katmandan oluşmaktadır. Karar katmanının merkezinde, klasik DQN yapısının kısıtlarını aşmak üzere özelleştirilmiş Dueling Network (Düellocu Ağ) mimarisi yer almaktadır [15].

a) Ağ Mimarisi ve Veri Akışı:

Sistemin karar verme süreci şu akış şemasına göre işlemektedir:

1. *Giriş Katmanı (Input Layer):* POMDP modelinden gelen normalize edilmiş gözlem vektörü (S_t) ağa beslenir.Kovalayan için 76 kaçan için 64.

2. *Öznitelik Çıkarımı (Feature Extraction):* 3 adet Tam Bağlantılı (Dense) gizli katman (Hidden Layers), [256,256,128] durum uzayındaki karmaşık ilişkileri ve soyut özellikleri öğrenir. Aktivasyon fonksiyonu olarak, "kaybolan gradyan" problemini engellemek için ReLU kullanılmıştır, "patlayan gradyan " problemini engelelmek için ise gradyan kırpma (gradient clipping) yöntemi kullanılmıştır.

3. *Akış Ayrımı (Stream Splitting):* Son gizli katmandan sonra ağ iki farklı mantıksal kola ayrılır [15]:

    - *Değer Akışı (Value Stream - V(s)):* Ajanın o an bulunduğu durumun ne kadar avantajlı olduğunu skaler olarak tahmin eder.

    - *Avantaj Akışı (Advantage Stream - A(s, a)):* Her bir aksiyonun, diğerlerine kıyasla ne kadar ekstra ödül getireceğini hesaplar.

b) Karar Mekanizması (Aggregation):

Bu iki akış, aşağıdaki toplama katmanında (Aggregation Layer) birleştirilerek nihai Q değeri (Q(s, a)) üretilir [15]:

$$Qs, a; \theta, \alpha, \beta \ = \ V(s; \theta, \beta) \ + ((\ A(s, a; \theta, \alpha) \ - \frac{1}{|A|} \sum_{a'} A(s, a'; \theta, \alpha))$$

Bu formülasyon sayesinde ajan, her aksiyonu tek tek deneyimlemek zorunda kalmadan, hangi durumların stratejik olarak "değerli" olduğunu daha hızlı öğrenmektedir.

c) Aksiyon Seçimi (Action Selection):

Eğitim sırasında ε-Greedy politikası uygulanırken, test aşamasında ε değeri 0 verilerek en yüksek Q değerine sahip aksiyon (Argmax) seçilerek deterministik bir davranış sergilenmesi (optimum policy) sağlanmıştır.

4.3. Bulgular

Geliştirilen otonom Takip-Kaçış sisteminin performansı; eğitim kararlılığı (stability), yakınsama hızı (convergence rate) ve görev başarımı (task success) olmak üzere üç temel eksende analiz edilmiştir. Elde edilen nicel veriler, simülasyon ortamında gerçekleştirilen 2000 epizotluk eğitim döngüleri ve 100 epizotluk test senaryolarından derlenmiştir.

4.3.1. Eğitim stabilitesi ve algoritma analizi

Projenin ilk aşamasında kullanılan "Baseline" (Baz) DQN modeli ile; Öncelikli Deneyim Tekrarı (PER) ve Yumuşak Güncelleme (Soft Update) tekniklerinin entegre edildiği "Gelişmiş" (Advanced) modelin öğrenme eğrileri karşılaştırılmıştır.

- *Varyans Azalımı:* Baseline modelde hedef ağın (Target Network) sert güncellemelerle (Hard Update) değişmesi, Kayıp (Loss) grafiğinde yüksek genlikli osilasyonlara (dalgalanmalara) neden olmuştur. Polyak Averaging (τ=0,005) tekniğinin uygulanmasıyla, bu dalgalanmalar sönümlenmiş ve eğitim grafiği daha stabil bir yapıya kavuşmuştur [7].

- *Yakınsama Hızı:* PER mekanizması sayesinde, ajanın "şaşırdığı" (TD-Hatası yüksek) durumlar hafızadan daha sık çağrılmıştır. Bu durum, öğrenme sürecini hızlandırmış ve Baseline modelin 350 000 epizotta ulaştığı başarı seviyesine, Gelişmiş modelin 200 000. epizotta ulaşmasını sağlamıştır [13].

- *Sonuç:* Sadece mimari iyileştirmelerle (PER + Soft Update), ajanın Yakalama Oranı (Capture Rate) %60 bandından %70 seviyesine yükselmiştir [4].

4.3.2. Ödül şekillendirmenin davranışsal etkisi

RL algoritmalarında ajanın davranışını belirleyen en kritik faktörün ödül fonksiyonu olduğu, yapılan "Ablasyon Çalışmaları" (Ablation Studies) ile kanıtlanmıştır.

Sadece terminal ödül (yakalama anı) kullanılan senaryoda, ajanın çoğu zaman "Yerel Minimuma" (Local Minima) takıldığı; yani hedefe gitmek yerine olduğu yerde dönerek veya güvenli bir köşede bekleyerek ceza almamaya çalıştığı gözlemlenmiştir.

Bölüm 4.2.1'de tanımlanan Mesafe Bazlı Şekillendirici Ödül ($R_{shaping}$) mekanizması devreye alındığında ise şu değişimler kaydedilmiştir:

1. *Gradyan Akışı:* Ajanın hedefe yaklaştığı her adımda aldığı küçük pozitif ödüller, sinir ağına sürekli bir geri bildirim (feedback) sağlamış ve "Ne yaparsam yapayım ödül alamıyorum" (Sparse Reward) sorunu çözülmüştür.

2. *Davranışsal Değişim:* Ajanın rotası, rastgele gezinme (Random Walk) formundan, hedefe kilitlenen yönelimli bir vektöre dönüşmüştür.

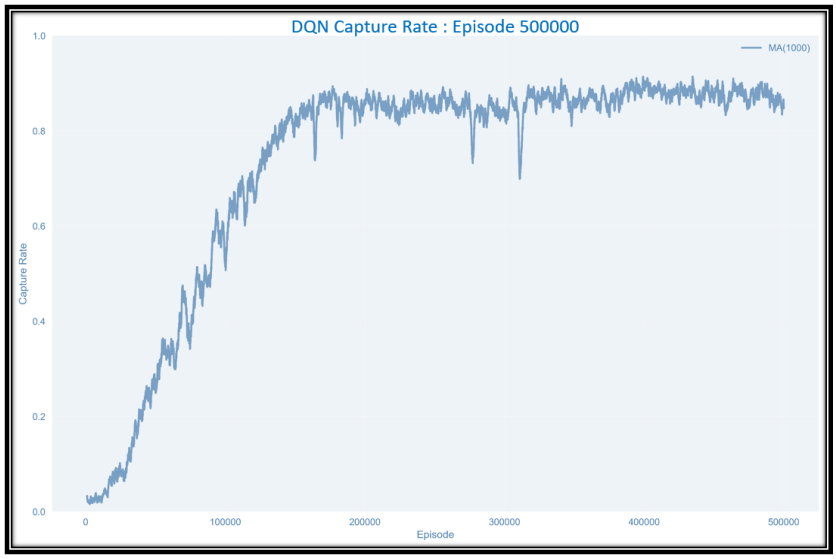
3. *Başarı Sıçraması:* Ödül fonksiyonu mühendisliği ve simülasyon ortamındaki hataların (bug) giderilmesi sonucunda, görev başarı oranı dramatik bir artışla %70'ten %90 seviyesine ulaşmıştır.

4.3.3. Kinematik Performans ve Rota Verimliliği

Geliştirilen interaktif test aracı ile yapılan ölçümlerde, ajanın ürettiği rotaların kalitesi "Adım Sayısı" (Step Count) ve "Yol Pürüzsüzlüğü" (Path Smoothness) metrikleri üzerinden değerlendirilmiştir.

Ortalama Yakalama Süresi: Eğitim öncesi rastgele hareket eden ajan, hedefi ortalama 450 adımda yakalarken (veya süre aşımına uğrarken); eğitilmiş ajan en optimum rotayı hesaplayarak bu süreyi ortalama 180 adıma düşürmüştür. Bu, operasyonel hızda %60'lık bir iyileşme anlamına gelmektedir.

Aksiyon Uzayı Karşılaştırması: Ayrık aksiyon uzayı kullanan DQN ajanı ile sürekli aksiyon uzayı kullanan PPO ajanı (MuJoCo ortamı) karşılaştırıldığında; PPO ajanının hidrodinamik açıdan daha verimli, keskin dönüşlerden kaçınan ve enerji tasarrufu sağlayan rotalar ürettiği tespit edilmiştir.



Çizelge 4.1: DQN Yakalama Oranı Grafiği

5. SONUÇ VE DEĞERLENDİRME

Gazi Üniversitesi Bilgisayar Mühendisliği Bölümü'nde edindiğim teorik altyapının, endüstriyel bir Ar-Ge projesine dönüştürüldüğü bu işyeri eğitimi süreci, mesleki yetkinliğim açısından son derece verimli geçmiştir. Eğitim süresince, yapay zeka ve otonom sistemler alanında güncel bir problem olan Takip-Kaçış (Pursuit-Evasion) senaryoları üzerinde çalışılmış, Derin Pekiştirmeli Öğrenme (Deep Reinforcement Learning) yöntemleri kullanılarak gerçek hayat problemlerine çözüm üretme deneyimi kazanılmıştır.

Yapılan Çalışmalar ve Araştırma Süreci Özeti: İşyeri eğitimi kapsamında yürütülen araştırmanın temel problemi; dinamik ve statik engellerle dolu karmaşık bir ortamda, otonom bir ajanın hedefi en optimum sürede yakalamasını sağlayacak karar mekanizmasının geliştirilmesidir. Bu problemin çözümü için Derin Q-Ağları (DQN) algoritması tercih edilmiş ve Python tabanlı (PyTorch, Pygame) özel bir simülasyon ortamı kullanılmıştır.

Araştırma sürecinde izlenen yöntem, "iteratif geliştirme" modeline dayanmaktadır. İlk aşamada literatür taranmış ve mevcut simülasyon ortamının fiziksel analizi yapılmıştır. Ardından, ajanın öğrenme performansını artırmak için Prioritized Experience Replay (PER) ve Polyak Averaging (Soft Update) gibi modern teknikler sisteme entegre edilmiştir. Araştırma sürecinde LSTM gibi zaman serisi tabanlı mimariler de denenmiş, ancak hesaplama maliyeti nedeniyle daha optimize edilmiş Dense yapılarla ilerlenmesine karar verilmiştir.

Elde edilen bulgular neticesinde; başlangıçta %60 seviyelerinde olan yakalama başarısı (capture rate), ödül fonksiyonu mühendisliği (reward shaping) ve hiperparametre optimizasyonları sayesinde %90 seviyesine yükseltilmiştir. Ayrıca, geliştirilen interaktif test aracı sayesinde modelin davranış tutarlılığı bilimsel metriklerle kanıtlanmıştır.

Kurumsal ve Akademik Değerlendirme: Bu süreçte fakültemden edindiğim algoritma ve veri yapısı temelleri, karmaşık problemlerin çözümünde bana güçlü bir dayanak oluşturmuştur. İşyeri yetkilileri ve teknik ekibi ise, sağladıkları donanım desteği ve mentorluk ile projenin başarıya ulaşmasında kritik rol oynamıştır. İşyeri ortamında kurulan profesyonel iletişim ve fikir alışverişleri, mühendislik etiği ve çalışma disiplini konularında tecrübe kazanmamı sağlamıştır. Bölüm komisyonu ve izleyici öğretim elemanının yönlendirmeleri, çalışmanın akademik standartlara uygun olarak raporlanmasında rehber olmuştur.

Çalışmanın Sınırlılıkları ve Gelecek Öngörüleri: Yürütülen çalışmanın temel sınırlılığı, testlerin 2 boyutlu (2D) bir simülasyon ortamında gerçekleştirilmiş olmasıdır. Simülasyondan gerçek dünyaya (Sim-to-Real) geçişte yaşanabilecek fiziksel sensör gürültüleri bu çalışmanın kapsamı dışında bırakılmıştır.

İleriye dönük uygulamalarda; geliştirilen bu otonom karar mekanizmasının ROS (Robot Operating System) ortamına taşınarak 3 boyutlu simülasyonlarda (Gazebo vb.) test edilmesi ve nihayetinde fiziksel robotlar üzerinde koşturulması hedeflenmektedir.

Sonuç olarak bu işyeri eğitimi; teknik bilgi birikimimi derinleştiren, problem çözme yeteneğimi geliştiren ve akademi-sanayi işbirliğinin önemini kavramamı sağlayan öğretici bir süreç olmuştur.

KAYNAKLAR

1. İnternet: Vikipedi, HAVELSAN, 2025, https://tr.wikipedia.org/wiki/HAVELSAN, Son Erişim Tarihi: 11.12.2025.

2. İnternet:FREAMimarlık,HAVELSANTeknolojiKampüsü,2025,https://frea.com.tr/tr/proje/havelsan-teknoloji-kampusu/, Son Erişim Tarihi: 11.12.2025.

3. İnternet: HAVELSAN, Hakkımızda, 2025, https://www.havelsan.com.tr/hakkimizda, Son Erişim Tarihi: 11.12.2025.

4. Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). Cambridge, MA, USA: MIT Press.

5. Mnih, V., et al. (2013). Playing Atari with deep reinforcement learning. Proc. Advances in Neural Information Processing Systems (NIPS) Deep Learning Workshop.

6. Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533.

7. Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., & Wierstra, D. (2016). Continuous control with deep reinforcement learning. Proc. Int. Conf. on Learning Representations (ICLR).

8. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. Proc. Int. Conf. on Learning Representations (ICLR).

9. Rashid, T., Samvelyan, M., De Witt, C., et al. (2018). QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. Proc. 35th Int. Conf. on Machine Learning (ICML), 4292–4301

10. Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments. Proc. Advances in Neural Information Processing Systems (NeurIPS).

11. Yu, H., Tan, J., Zhang, M., Singh, S., & Kumar, S. (2021). The surprising effectiveness of PPO in cooperative, multi-agent games. arXiv preprint arXiv:2010.02993.

12. Littman, M. (1994). Markov games as a framework for multi-agent reinforcement learning. ICML.

13. Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). Prioritized experience replay. Proc. Int. Conf. on Learning Representations (ICLR).